

PFMA – Features & Data Structure Implementation

Every feature mapped to its underlying data structure

+ Add Transaction (Expense / Income)

SINGLY LINKED LIST · TAIL INSERTION $O(N)$

Each new transaction becomes a `Transaction` node allocated on the heap with `new`. The `TransactionList::add()` walks to the end of the chain (following `next` pointers) and attaches the new node — classic tail insertion. The undo `stack` gets a snapshot pushed via `push()` (LIFO). The `CategoryTree` BST and `DateTree` are then rebuilt from the linked list to stay synchronized.

💰 Dynamic Balance, Income & Expenses Tabs

LINKED LIST TRAVERSAL $O(N)$ · RECALCULATED EVERY REPAINT

Three functions — `getTotalExpenses()`, `getTotalIncome()`, `getBalance()` — all traverse the full linked list. Starting from `head`, each follows `next` pointers to the end, summing `amount` where `type==1` (expense) or `type==0` (income). Called on every `WM_PAINT`, so numbers update instantly after any add/undo/redo. Tab widths auto-resize based on digit count to prevent overlap.

🥧 Pie Chart — Category Distribution

BST DFS INORDER TRAVERSAL · SORTED OUTPUT $O(N)$

Calls `CategoryTree::getDFSList()` which runs `inorderDFS()` — a recursive Left→Root→Right traversal. Because the BST is ordered alphabetically, categories come out **sorted A–Z** without any sorting algorithm. Each category's share of total expenses determines its pie slice angle. 8 vibrant colors for visual distinction.

📊 Analytics Bar Chart

BST DFS INORDER · SAME SORTED DATA AS PIE CHART

Uses the exact same `getDFSList()` DFS inorder result. Bars are scaled relative to the highest-spending category. Grid lines drawn at 25%, 50%, 75%, 100% of maximum. Category names and dollar amounts displayed below and above each bar.

↶ Undo & ↷ Redo Buttons

TWO EXPLICIT LIFO STACKS · `PUSH()` / `POP()` / `TOP()` / `EMPTY()`

Undo: Checks `undoStack.empty()` (underflow protection). Pops the top record (`pop()` $O(1)$), removes the last linked list node, pushes the record onto `redoStack`. Rebuilds both trees.
Redo: Pops from `redoStack`, re-calls `addTransaction()` with the stored data. Adding a new transaction **clears redoStack** — same behavior as Ctrl+Z in all software.
Both stacks use `std::stack<UndoRecord>` where `UndoRecord` stores a snapshot of all 5 transaction fields.

📄 Export Monthly Report (.txt)

BFS ON DATETREE · QUEUE FIFO · MAP AGGREGATION

Creates `expenses_MM_YYYY.txt` with all transactions (linked list traversal). Then runs **31 BFS searches** — one per day via `DateTree::searchByDateBFS()`. Each BFS call uses a `queue<DateNode*>` to traverse Month→Day→Transaction levels. Expense totals are aggregated per category using `map<string,double>`. The highest total is written as "Most Spent Category" in the report.

Search by Category

DFS INORDER ON BST

- 1 User types a search term (e.g. "foo")
- 2 `CategoryTree::searchDFS()` calls `inorderDFSSearch()`
- 3 → DFS starts at root, goes **all the way left** (smallest category)
- 4 → At **each node** : converts category to lowercase, checks if `searchTerm` is a substring
- 5 → If match: adds `{category, totalAmount}` to results
- 6 → Continues DFS: then goes **right** (larger categories)
- 7 Results displayed **alphabetically sorted** — free bonus of BST + inorder
- 8 Individual transactions from linked list also shown for matched categories

Search by Date (DD / MM / YYYY)

BFS LEVEL-ORDER ON DATETREE

- 1 User enters Day, Month, Year in separate fields
- 2 `DateTree::searchByDateBFS()` is called
- 3 → **Queue initialized** with root node (Level 0 = month)
- 4 → **Level 0 check:** dequeues month node — if month matches, enqueues ALL day children. If not, skips (prunes branch)
- 5 → **Level 1 check:** dequeues each day node — if day matches, enqueues ALL transaction children
- 6 → **Level 2 collection:** dequeues transaction leaf nodes — year is checked, matching transactions are collected
- 7 Queue empty → BFS complete. Results = all transactions on that exact date
- 8 Display shows count + each transaction's details



All Data Structures at a Glance

Singly Linked List

LINEAR STRUCTURES · TRANSACTION STORAGE

All transactions. `Transaction* next` links nodes. Head pointer. Tail insertion O(n). Traversal for all balance calculations.

Stack (Explicit LIFO)

STACK · UNDO/REDO

`undoStack` + `redoStack` of `UndoRecord` . `push()` on add, `pop()` on undo. Underflow checked before every pop.

Stack (Implicit LIFO)

STACK + TREES · DFS CALL STACK

DFS recursion = call stack in action. Each `inorderDFS()` call pushes a frame; stack unwinds as recursion returns.

Queue (FIFO)

QUEUE · BFS DATE SEARCH

`queue<DateNode*>` in `searchByDateBFS()` . Enqueues root, dequeues level by level. Guarantees Month→Day→Transaction order.

Binary Search Tree (BST)

TREES · CATEGORYTREE

Categories as keys (alphabetical). Left < Root < Right. Recursive insert with aggregation. Rebuilt from linked list on every expense add.

DFS Inorder Traversal + Search

TREES · CHARTS + CATEGORY SEARCH

`inorderDFS()` for sorted pie/bar charts. `inorderDFSSearch()` for category search — visits every node, substring match, results alphabetically sorted.

3-Level DateTree + BFS

TREES · MONTH→DAYS→TRANSACTIONS

Root=month, Level1=days, Level2=transactions. Built from linked list. `searchByDateBFS()` uses Queue for level-order search + export analysis.

BFS-Powered Export Analysis

TREES + QUEUE · 31 BFS CALLS · MOST SPENT CATEGORY

Loops days 1–31, calls `searchByDateBFS()` each time. Map aggregates expense totals per category. Maximum found → written to .txt export.